UML Diagrams

g-code-viewer

Black Sphere Industries

Generated 2026-03-23

Contents:

[OK] =

[text] SEQUENCE DIAGRAM: Parse and Visualize G-Code

[OK] SEQUENCE DIAGRAM: Simulation Playback

[OK] SEQUENCE DIAGRAM: File Upload

[OK] COMPONENT DIAGRAM

[text] STATE DIAGRAM

=

SimulationController
+play() : void
+pause() : void
+stop() : void
+stepForward() : void
+stepBack() : void
+setSpeed(speed) : void
+updateDisplay() : void
+animationLoop() : void
ExamplePresider
+getExample(name) : string
+enlarge(n) : string
+circle(r) : string
+contain() : string
triggers render
triggers render
manifies
Canvas2DRenderer
+canvas HTMLCanvasElement
+ctx CanvasRenderingContext2D
+panX float
+panY float
+zoom float
+dpr float
+setup() : void
+handleZoom(e) : void
+handlePan(e) : void
+zoomToFit() : void
+render() : void
+drawGrid(ctx) : void
+drawNaves(ctx) : void
+drawBoundingBox(ctx) : void
Canvas3DRenderer
+canvas HTMLCanvasElement
+ctx CanvasRenderingContext2D
+rotX float
+rotY float
+zoom float
+dpr float
+setup() : void
+handleRotation(e) : void
+handleZoom(e) : void
+render() : void
+project(x, y, z) : ScreenPoint
+drawNaves(ctx) : void
+drawAxes(ctx) : void
GCodeParser
+units string
+feedRate string
+plane string
+spindle boolean
+spindleDir int
+positions PositionMap
+offsets OffsetMap
+feedRate float
+spindleSpeed float
+tool int
+moves Move[]
+parse(gcodeText) : Move[]
+parseLineCommands(line) : string[]
+processCommand(cmd, state) : void
+processGCode(value, state) : void
+processMCode(value, state) : void
reads
reads
creates array of
AppState
+gcode Move[]
+moves ProcessedMove[]
+currentIdx int
+isPlaying boolean
+isPaused boolean
+speed float
+toolDiameter float
+units string
+currentX float
+currentY float
+currentZ float
+currentA float
+currentB float
+currentC float
+currentT int
+feedRate string
+plane string
+offsetX Float
+offsetY Float
+offsetZ Float
+bounds BoundingBox
+stats StatsInfo
shares array of
contains
contains
Move
+line int
+type string
+command string
+rapid boolean
+gcode int
+mcode int
+x Float
+y Float
+z Float
+i float
+j float
+k Float
+r float
+feedRate float
+spindleSpeed float
+tool int
+arcDir int
+toolComp string
+spindle string
processed into
BoundingBox
+minX Float
+maxX Float
+minY Float
+maxY Float
+minZ Float
+maxZ float
StatsInfo
+totalDist float
+cutDist float
+rapidDist float
+toolChanges int
+machineTime float
+feedMin Float
+feedMax Float
+totalCommands int
ProcessedMove
+startX Float
+startY Float
+startZ Float
+endX float
+endY float
+endZ float
+feedRate float
+spindleSpeed float
+tool int
+type string
+arcLength float

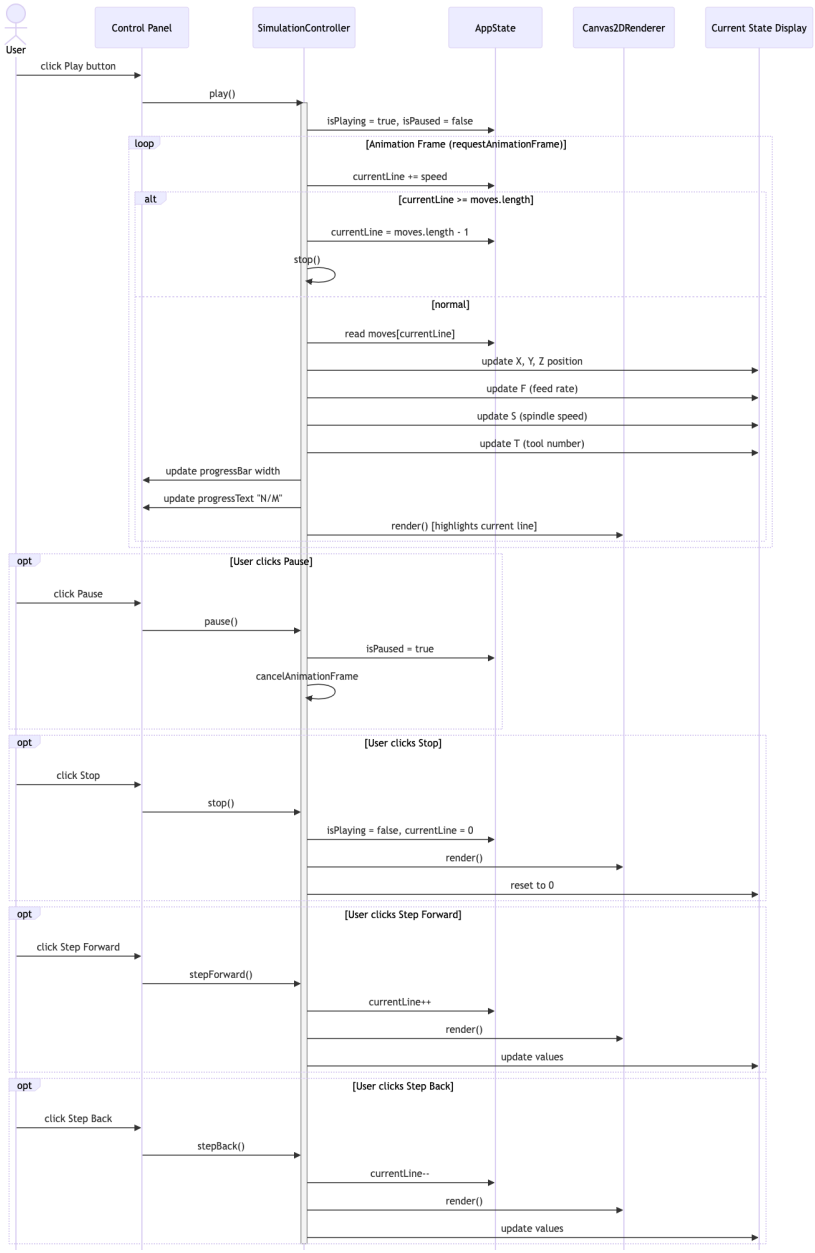
SEQUENCE DIAGRAM: Parse and Visualize G-Code

```
sequenceDiagram
    actor User
    participant UI as Editor Panel
    participant Parser as GCodeParser
    participant Proc as MoveProcessor
    participant State as AppState
    participant R2D as Canvas2DRenderer
    participant R3D as Canvas3DRenderer
    participant Stats as Statistics Panel

    Note over User,Stats: G-Code Input (Paste Tab)
    User->>UI: paste G-code text into textarea
    User->>UI: click "Parse & Render"

    activate Proc
    Proc->>Parser: new GCodeParser()
    activate Parser
    Proc->>Parser: parse(gcodeText)

    loop For each line in G-code
        Parser->>Parser: split line on ; or ( to remove comments
        Parser->>Parser: parseLineCommands(code)
        Parser->>Parser: match tokens [G/M codes, X/Y/Z/I/J/K/F/S/T values]

        loop For each token
            alt G-code
                Parser->>Parser: processGCode(value, move)
                Note right of Parser: G0=rapid, G1=linear,<br/>G2/G3=arc CW/CCW,<br/>G17/18/19=plane,<br/>G20/21=units,<br/>G90/91
            else M-code
                Parser->>Parser: processMCode(value, move)
                Note right of Parser: M3/4=spindle on,<br/>M5=spindle off,<br/>M6=tool change,<br/>M30=program end
            else Coordinate
                Parser->>Parser: store x/y/z/i/j/k/r/f/s/t
            end
        end

        opt move has type
            Parser->>Parser: push move to moves[]
        end
    end

    Parser-->>Proc: rawMoves[]
    deactivate Parser

    Proc->>State: state.gcode = rawMoves

    loop For each rawMove
        Proc->>Proc: create processedMove with start position
        alt absolute mode
            Proc->>Proc: newX/Y/Z = move.x/y/z or current
        else incremental mode
            Proc->>Proc: newX/Y/Z = current + move.x/y/z
        end

        alt linear or rapid
            Proc->>Proc: compute distance = sqrt(dx^2 + dy^2 + dz^2)
            Proc->>Proc: accumulate totalDist, cutDist or rapidDist
```

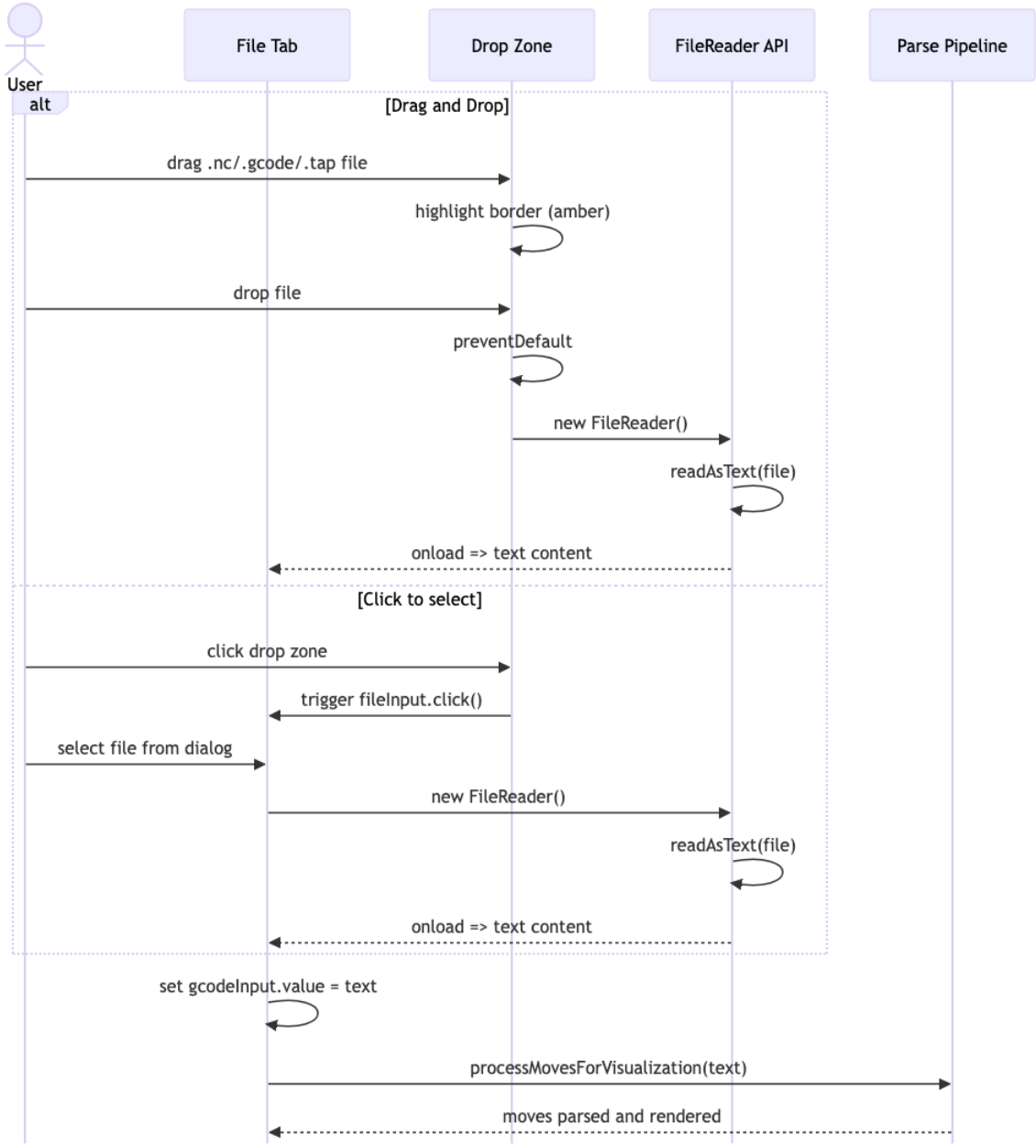
```
            opt feedRate > 0
                Proc->>Proc: machineTime += (dist / feedRate) * 60
            end
        else arc
            Proc->>Proc: calculateArcLength(x1,y1,x2,y2,i,j,r)
            alt R format
                Proc->>Proc: angle = 2*asin(d/|r|), length = |r|*angle
            else I,J format
                Proc->>Proc: compute center, angles, length
            end
            Proc->>Proc: accumulate distances
        else toolchange
            Proc->>Proc: toolChanges++
        end

        Proc->>Proc: update bounds (min/max X,Y,Z)
        Proc->>Proc: update current position
        Proc->>Proc: push to processed moves[]
    end

    Proc->>State: state.moves = processedMoves
    Proc->>State: state.stats = computed statistics
    deactivate Proc
```

... (107 lines total - see .mmd source)

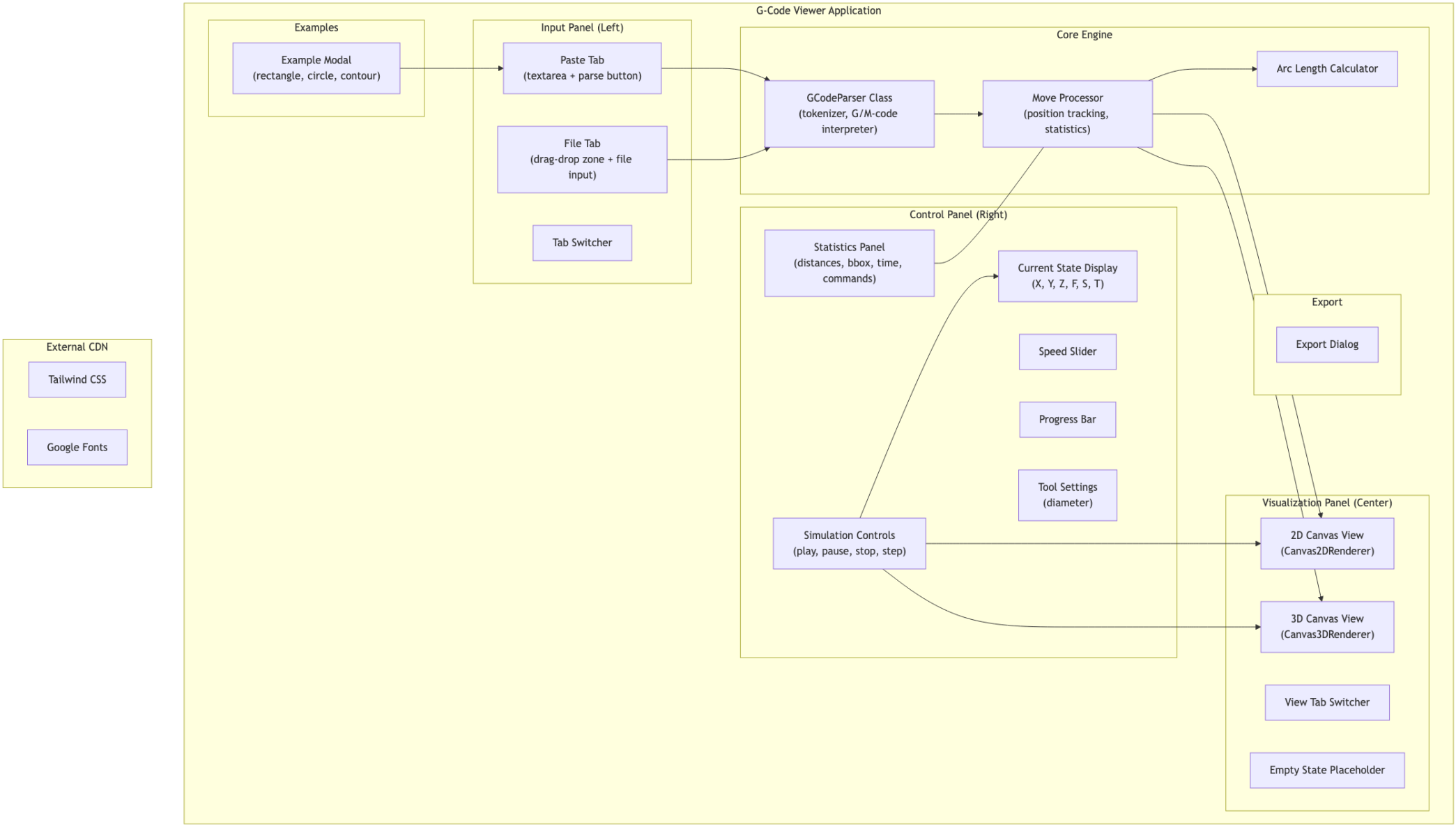
SEQUENCE DIAGRAM: Simulation Playback



SEQUENCE DIAGRAM: File Upload



COMPONENT DIAGRAM



STATE DIAGRAM

```
stateDiagram-v2
    [*] --> Empty

    state "Empty" as Empty {
        [*] --> NoData
        NoData : No G-code loaded
        NoData : Canvas shows placeholder
    }

    state "InputMethod" as InputMethod {
        [*] --> PasteMode
        PasteMode --> FileMode : click File tab
        FileMode --> PasteMode : click Paste tab
        PasteMode --> Parsing : click Parse & Render
        FileMode --> FileLoading : drop file or select file
        FileLoading --> Parsing : FileReader.onload
    }

    state "Parsed" as Parsed {
        [*] --> ViewSelection
        state ViewSelection {
            [*] --> View2DActive
            View2DActive --> View3DActive : click 3D tab
            View3DActive --> View2DActive : click 2D tab
        }

        state "Simulation" as SimState {
            [*] --> SimStopped
            SimStopped --> SimPlaying : play()
            SimPlaying --> SimPaused : pause()
            SimPaused --> SimPlaying : play()
            SimPlaying --> SimStopped : stop() or reached end
            SimPaused --> SimStopped : stop()

            SimStopped --> StepMode : step forward/back
            StepMode --> SimStopped : idle
            StepMode --> SimPlaying : play()
        end
    }

    Empty --> InputMethod : user starts input
    InputMethod --> Empty : example selected
    Parsing --> Parsed : parse successful
    Parsed --> InputMethod : modify and re-parse

    state "SpeedControl" as SpeedCtrl {
        [*] --> Speed1x
        Speed1x --> SpeedCustom : adjust slider
        SpeedCustom --> Speed1x : reset
    }

    Parsed --> SpeedCtrl : adjust speed
```